The operations on `std::atomic_flag` buck the trend, in that they spell out the "flag" part in the names: `std::atomic_flag_test_and_set()`, `std::atomic_flag_clear()`, although the additional variants that specify the memory ordering again have the `_explicit` suffix: `std::atomic_flag_test_and_set_explicit()` and `std::atomic_flag_clear_explicit()`.

The C++ Standard Library also provides free functions for accessing instances of `std::shared_ptr<>` in an atomic fashion. This is a break from the principle that only the atomic types support atomic operations, because `std::shared_ptr<>` is quite definitely *not* an atomic type. However, the C++ Standards Committee felt it was sufficiently important to provide these extra functions. The atomic operations available are *load*, *store*, *exchange*, and *compare/exchange*, which are provided as overloads of the same operations on the standard atomic types, taking a `std::shared_ptr<>*` as the first argument:

```
std::shared_ptr<my_data> p;
void process_global_data()
{
    std::shared_ptr<my_data> local=std::atomic_load(&p);
    process_data(local);
}
void update_global_data()
{
    std::shared_ptr<my_data> local(new my_data);
    std::atomic_store(&p,local);
}
```

As with the atomic operations on other types, the `_explicit` variants are also provided to allow you to specify the desired memory ordering, and the `std::atomic_is_lock_free()` function can be used to check whether the implementation uses locks to ensure the atomicity.

As described in the introduction, the standard atomic types do more than just avoid the undefined behavior associated with a data race; they allow the user to enforce an ordering of operations between threads. This enforced ordering is the basis of the facilities for protecting data and synchronizing operations such as `std::mutex` and `std::future<>`. With that in mind, let's move on to the real meat of this chapter: the details of the concurrency aspects of the memory model and how atomic operations can be used to synchronize data and enforce ordering.

5.3. Synchronizing operations and enforcing ordering

Suppose you have two threads, one of which is populating a data structure to be read by the second. In order to avoid a problematic race condition, the first thread sets a flag to indicate that the data is ready, and the second thread doesn't read the data until the flag is set. The following listing shows such a scenario.

Listing 5.2. Reading and writing variables from different threads